Kops setup
Kubernetes Cluster Setup using KOPS

Chapter 1 - Introduction to KOPS

What is KOPS?

KOPS (Kubernetes Operations) is an official Kubernetes project used to create, manage, upgrade, and delete production-grade Kubernetes clusters on cloud platforms.

Unlike Minikube, which creates only a single-node cluster for learning, KOPS creates a real multi-node Kubernetes cluster in AWS.

KOPS automates:

- EC2 Instance creation
- VPC configuration
- Security Groups
- IAM Roles
- Auto Scaling Groups
- Route53 DNS
- Kubernetes Installation
- Cluster Upgrades
- Cluster Validation

Because of this automation, KOPS is considered one of the easiest ways to deploy a production-ready Kubernetes cluster on AWS.

Why Do We Use KOPS?

Instead of manually installing Kubernetes on every EC2 instance using kubeadm, KOPS performs all the setup automatically.

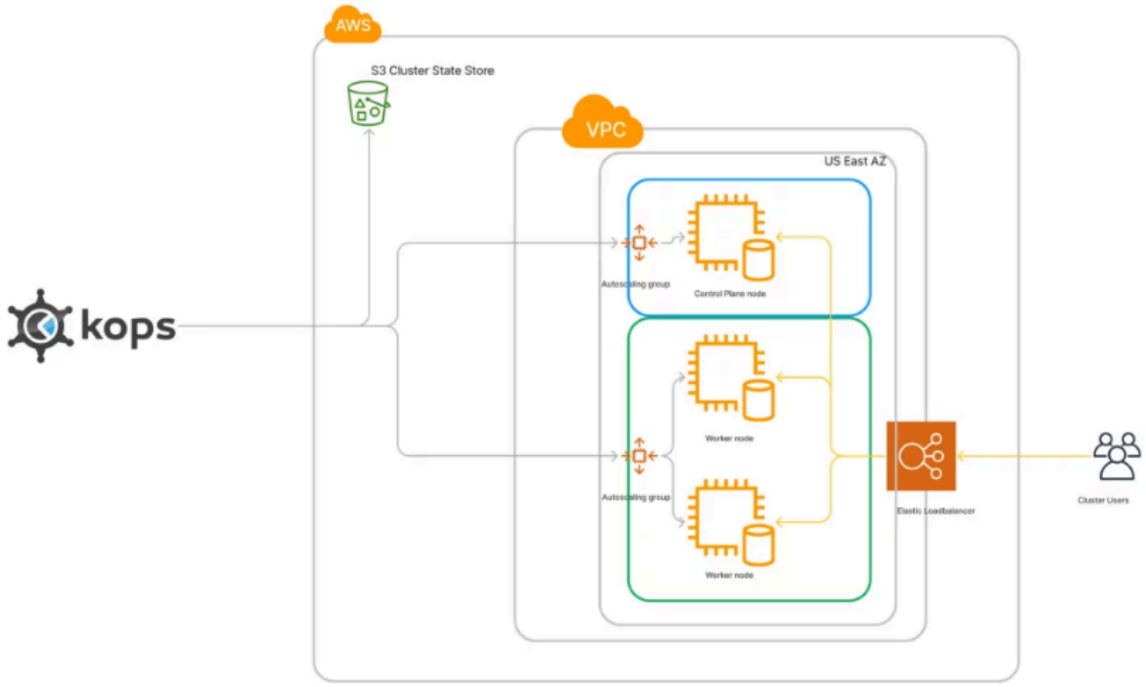
Advantages include:

- Automated Kubernetes installation
- High Availability support
- Automatic certificate generation
- Easy upgrades
- Easy scaling
- Easy cluster deletion
- Production-ready architecture

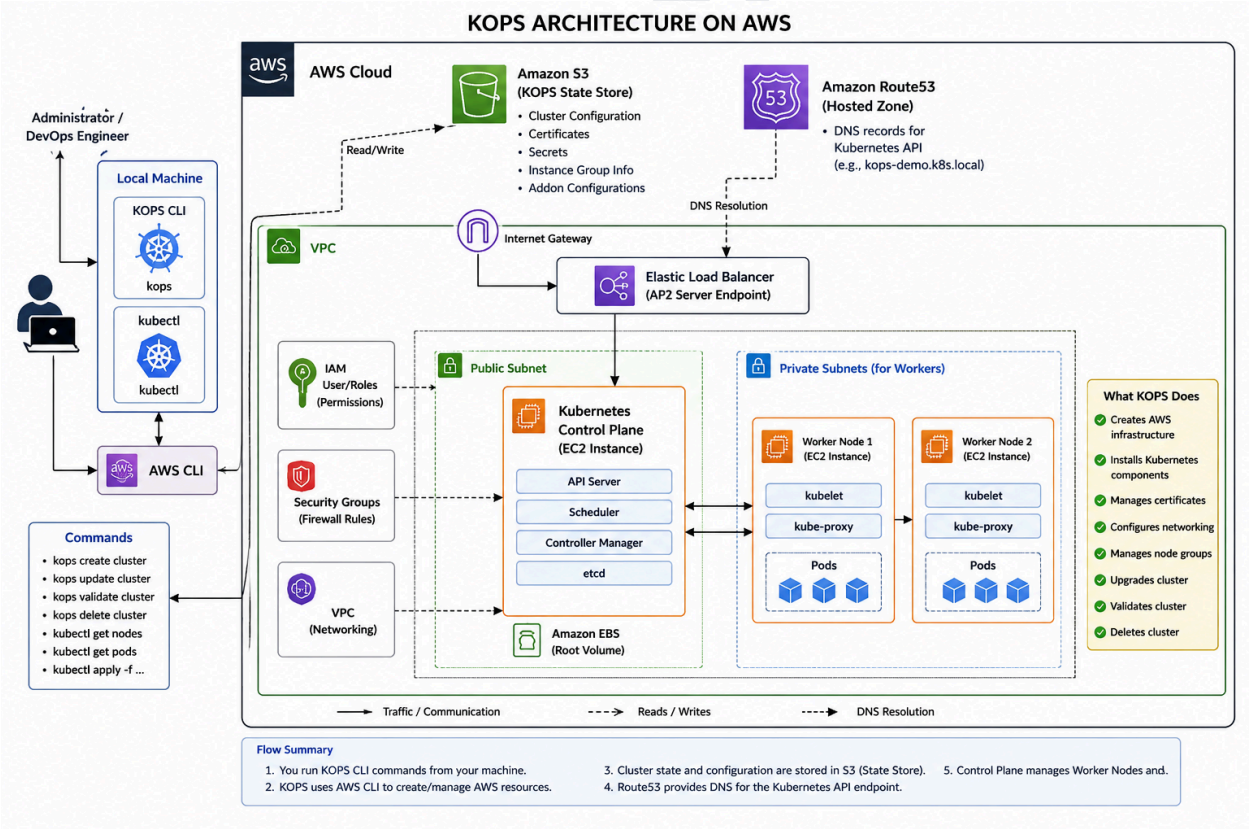
Difference Between Minikube and KOPS

Minikube	**KOPS**
Local machine	AWS Cloud
Single Node	Multi Node
Learning	Production
No High Availability	Supports HA
Runs in Virtual Machine	Runs on EC2

Chapter2 :KOPS Architecture



kOps-based Kubernetes cluster architecture



What is KOPS Architecture?

KOPS Architecture refers to the collection of AWS services, Kubernetes components, and KOPS automation that work together to create and manage a Kubernetes cluster.

Unlike Minikube, where Kubernetes runs on a single local machine, KOPS deploys a distributed Kubernetes cluster in AWS.

When a user runs a KOPS command, KOPS communicates with AWS services to provision infrastructure, configure networking, install Kubernetes components, and prepare the cluster for application deployment.

KOPS

KOPS is a command-line tool that creates, manages, upgrades, validates, and deletes production-grade Kubernetes clusters on AWS.

Why Was KOPS Introduced?

Before KOPS, administrators had to:

- Launch EC2 instances
- Install Docker
- Install Kubernetes
- Configure etcd
- Configure certificates
- Configure networking
- Configure security
- Configure load balancers
- Configure IAM roles

This process was complex and time-consuming.

KOPS automates these tasks with a few commands.

Features of KOPS

- Automated cluster creation
- Production-ready deployment
- High Availability support
- Automatic TLS certificates
- Automatic etcd setup
- Automatic worker node configuration
- Rolling updates
- Easy upgrades
- Easy scaling
- Easy deletion

Advantages of KOPS

- Official Kubernetes project
- Reduces manual work
- Uses Infrastructure as Code concepts
- Supports multiple Availability Zones
- Easy to manage
- Ideal for production environments
- Supports cluster upgrades
- Integrates with AWS services

Limitations of KOPS

- Primarily designed for AWS
- Requires Route53 DNS
- Requires an S3 bucket as a state store
- AWS resources incur charges
- Slightly more complex than Minikube

Chapter 3- Prerequisites

1.AWS Account

An AWS account is required because KOPS provisions EC2 instances, VPCs, IAM roles, Route53 DNS, and S3 buckets.

Ensure billing is enabled and you have permissions to create these resources.

2.IAM User

Instead of using the root account, create an IAM user with `AdministratorAccess` (or a least-privilege policy suitable for KOPS).

Why not use the root account?

- Better security
- Easier auditing
- Supports key rotation
- Follows AWS best practices

---

3. INSTALL AWS CLI

What is AWS CLI?

AWS CLI (Command Line Interface) is a tool that allows you to manage AWS services directly from the terminal.

KOPS itself does not directly authenticate with AWS. Instead, it uses the AWS CLI credentials configured on the system.

Responsibilities

- Create EC2 instances
- Create IAM roles
- Create Security Groups
- Create Load Balancers
- Create VPC resources
- Access S3 buckets
- Manage Route53 records

Configure AWS CLI

```
aws configure
```

You will be prompted for:

```
AWS Access Key ID:
AWS Secret Access Key:
Default Region:
Default Output Format:
```

Verify Configuration

```
aws sts get-caller-identity
```

This command confirms that your credentials are valid and displays the AWS account ID and IAM user/role currently in use.

---

4. Install kubectl

What is kubectl?

`kubectl` is the Kubernetes command-line tool used to interact with the Kubernetes API server.

Common tasks include:

- Creating deployments
- Listing nodes
- Viewing pods
- Scaling applications
- Deleting resources
- Viewing logs

Download the latest stable version

```
curl -LO "https://dl.k8s.io/release/$(curl -L -s https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"
```

Make it executable

```
chmod +x kubectl
```

Move it to the system path

```
sudo mv kubectl /usr/local/bin/
```

Verify installation

```
kubectl version --client
```

5.Install KOPS

What is KOPS?

KOPS (Kubernetes Operations) is an official Kubernetes cluster management tool.

It automates the deployment and management of Kubernetes clusters on AWS.

Without KOPS, administrators would manually:

- Launch EC2 instances
- Install Kubernetes
- Configure etcd
- Configure certificates
- Configure networking
- Configure load balancers

KOPS automates all of these tasks.

Download KOPS (v1.35.1)

```
wget
https://github.com/kubernetes/kops/releases/download/v1.35.1/kops-linux-amd64
```

Make Executable

```
chmod +x kops-linux-amd64
```

Rename and move it

```
sudo mv kops-linux-amd64 /usr/local/bin/kops
```

Move Binary

`sudo mv kops-linux-amd64 /usr/local/bin/kops`.

Verify Installation

```
kops version
```

---

4. Amazon S3 (State Store)

What is a State Store?

A State Store is the central storage location where KOPS stores the complete cluster configuration.

Think of it as the **brain** of the KOPS cluster configuration.

KOPS reads from and writes to this bucket whenever changes are made to the cluster.

Information Stored in the State Store

- Cluster configuration
- Node configuration
- Kubernetes certificates
- Encryption keys
- Secrets
- ETCD configuration
- Networking configuration
- Instance group configuration
- Add-on configuration

Example configuration:

```
export KOPS_STATE_STORE=s3://kops-demopurpose.k8s.local
```

Verify:

```
echo $KOPS_STATE_STORE
```

Expected output:

```
s3://kops-demopurpose.k8s.local
```

---

5. Amazon Route53

What is Route53?

Amazon Route53 is AWS's managed DNS (Domain Name System) service.

KOPS uses Route53 to create DNS records for the Kubernetes API Server endpoint.

Instead of connecting to an IP address, clients connect using a DNS name.

Example:

```
api.kops-demopurpose.k8s.local
```

Why is Route53 Required?

- Resolves the Kubernetes API endpoint.
- Allows worker nodes to locate the control plane.
- Supports certificate generation.
- Simplifies API access.

Without Route53, KOPS cannot create a functional cluster using its default DNS configuration.

---

6. Amazon VPC

A Virtual Private Cloud (VPC) is an isolated network in AWS where all Kubernetes resources are deployed.

The VPC provides:

- Private networking
- IP address management
- Internet connectivity
- Security isolation

A typical KOPS deployment creates:

- Public subnets
- Private subnets (optional, depending on topology)
- Route tables
- Internet Gateway
- NAT Gateway (in private topologies)

---

7. Security Groups

Security Groups act as virtual firewalls for EC2 instances.

They control inbound and outbound traffic.

Example rules:

Port	Protocol	Purpose
22	TCP	SSH
443	TCP	Kubernetes API
80	TCP	HTTP
30000–32767	TCP	NodePort Services

---

8. Elastic Load Balancer (ELB)

KOPS automatically creates an Elastic Load Balancer for the Kubernetes API Server.

Responsibilities:

- Receives API requests.
- Distributes traffic to the control plane.
- Provides a single endpoint for `kubectl`.
- Improves availability.

---

9. Kubernetes Control Plane

The Control Plane is the **brain of the Kubernetes cluster**.

It makes all cluster decisions.

Components:

API Server

Receives all Kubernetes requests.

Scheduler

Assigns Pods to suitable worker nodes.

Controller Manager

Maintains the desired state of the cluster.

etcd

Stores the cluster state and metadata.

---

10. Worker Nodes

Worker Nodes execute application workloads.

Each worker node contains:

- kubelet
- kube-proxy
- Container Runtime (containerd)
- Pods

The Control Plane continuously monitors and manages these nodes.

---

11. Pods

Pods are the smallest deployable units in Kubernetes.

A Pod contains one or more containers that share:

- Network
- Storage
- Lifecycle

Applications always run inside Pods.

KOPS Cluster Creation Workflow

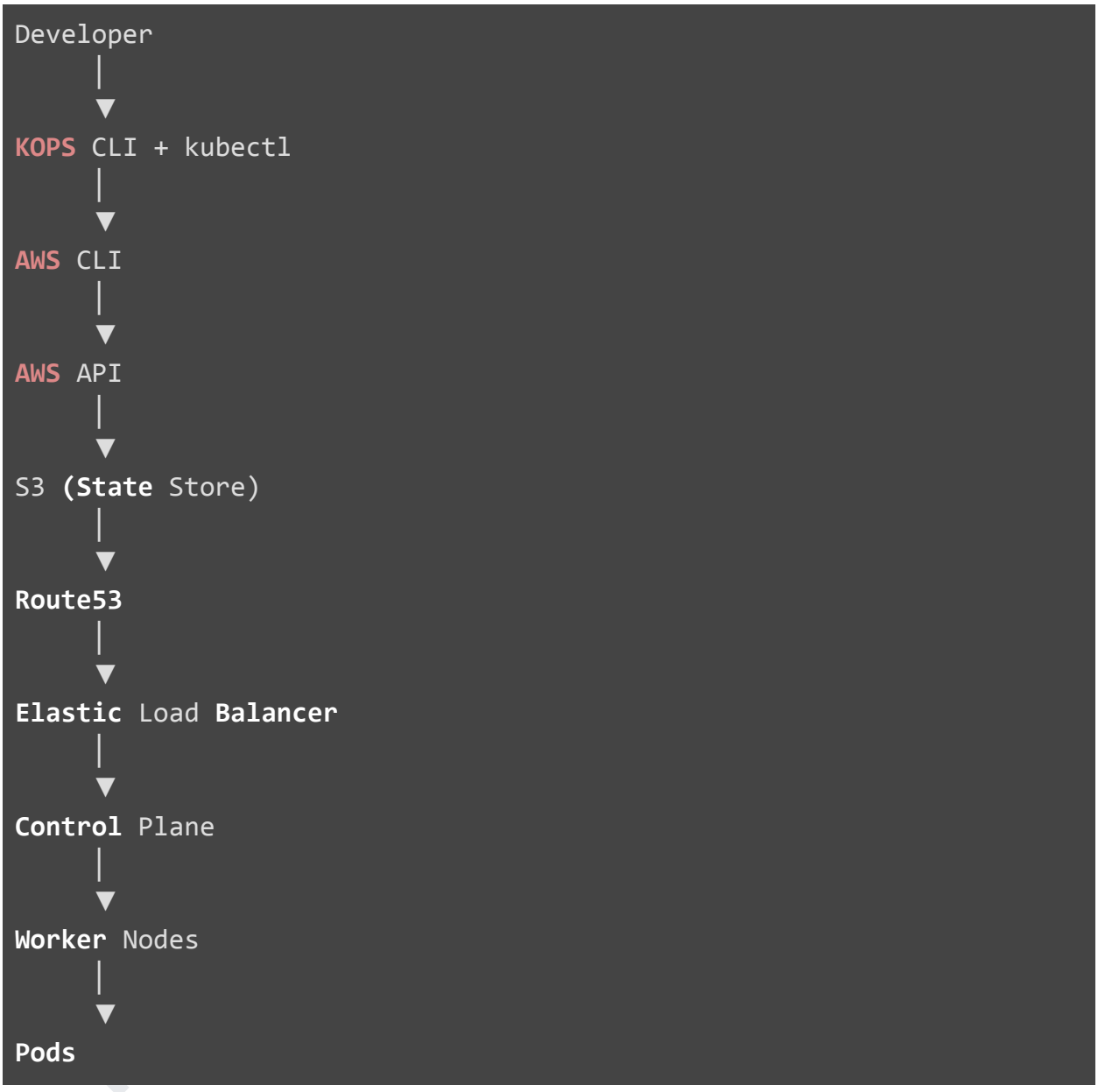
When you run:

```
kops create cluster \
--name kops-demopurpose.k8s.local \
--zones us-east-1a \
--control-plane-count=1 \
--node-count=2
```

The following sequence occurs:

1. KOPS reads AWS CLI credentials.
2. KOPS connects to the configured S3 State Store.
3. KOPS writes the cluster configuration to the S3 bucket.
4. KOPS verifies the Route53 hosted zone.
5. KOPS generates Kubernetes certificates.
6. KOPS creates the VPC (if required).
7. KOPS creates Security Groups.
8. KOPS creates the Control Plane EC2 instance.
9. KOPS creates Worker Node EC2 instances.
10. KOPS configures the Elastic Load Balancer.
11. Kubernetes components are installed on all nodes.
12. Worker Nodes join the cluster.
13. The Control Plane validates the cluster.
14. The cluster becomes ready for workloads.

Architecture Summary



Chapter 4: AWS Infrastructure Preparation for KOPS

Learning Objectives

- Understand the AWS resources required before creating a KOPS cluster.
- Create an S3 bucket for the KOPS State Store.
- Create a Route53 Hosted Zone.
- Understand the purpose of IAM, VPC, Security Groups, EC2 Key Pair, and DNS.
- Verify that the AWS environment is ready for KOPS.

Introduction

Before KOPS can create a Kubernetes cluster, AWS must already contain some required resources.

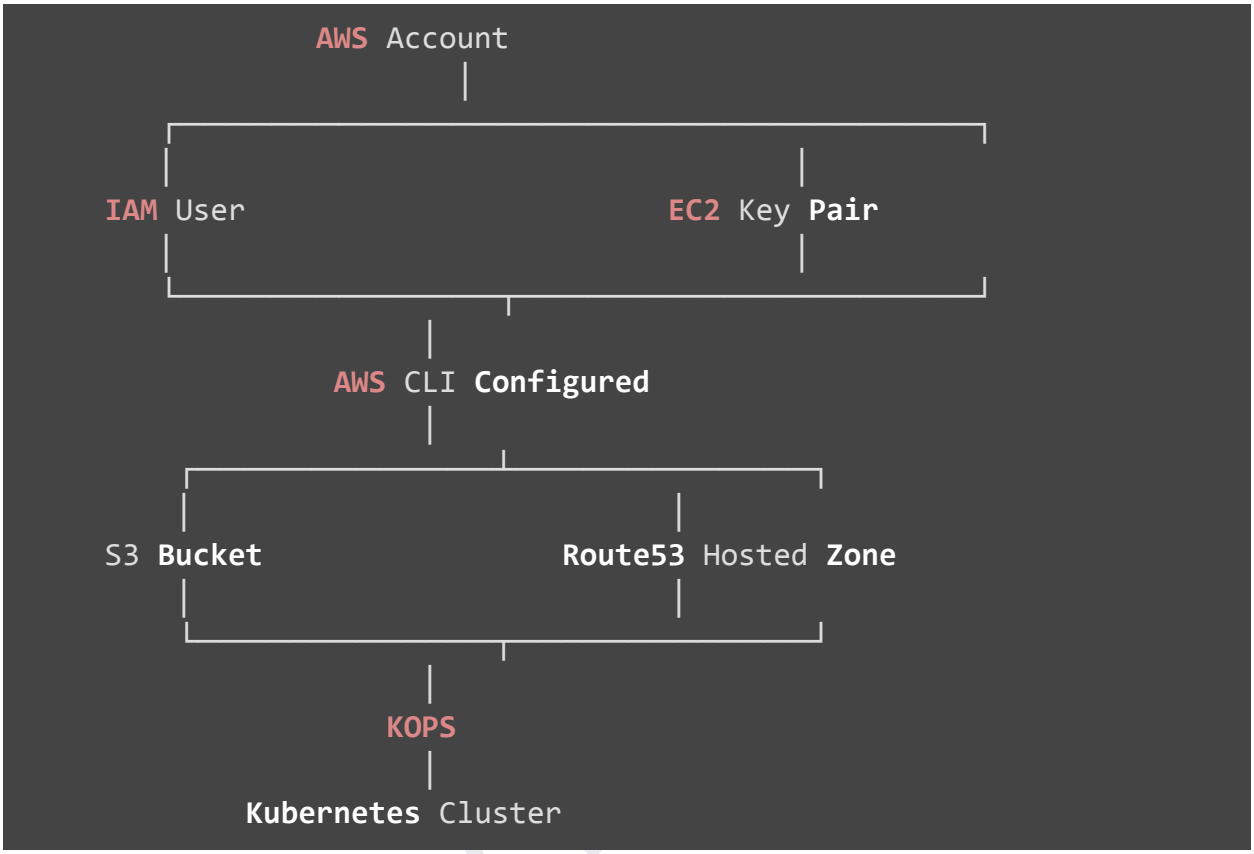
Unlike Minikube, KOPS creates a real production cluster inside AWS. Therefore, AWS infrastructure must be prepared beforehand.

The two most important resources are:

- Amazon S3 Bucket (State Store)
- Amazon Route53 Hosted Zone

Without these resources, KOPS cannot create a cluster.

AWS Infrastructure Required



AWS Resources Required

AWS Resource	Purpose
AWS Account	Hosts all cloud resources
IAM User	Authentication and permissions
AWS CLI	Communicates with AWS
S3 Bucket	Stores cluster state
Route53	DNS for Kubernetes API

EC2 Key Pair	SSH access to nodes
VPC	Networking
Security Groups	Firewall
EC2 Instances	Master and Worker Nodes

---

1. AWS Account

A valid AWS account is required because KOPS provisions infrastructure such as:

- EC2 Instances
- VPC
- Route53
- IAM Roles
- Elastic Load Balancer
- Auto Scaling Groups
- Security Groups
- EBS Volumes

Without an AWS account, KOPS cannot create any infrastructure.

---

2. IAM User

Why IAM?

IAM (Identity and Access Management) controls permissions within AWS.

KOPS requires permissions to create and manage AWS resources.

Best Practice

Never use the AWS Root Account for daily work.

Instead:

- Create an IAM User
- Assign AdministratorAccess (for lab environments)
- Generate an Access Key and Secret Key

IAM Permissions Required by KOPS

KOPS requires permissions for:

- EC2
- S3
- Route53
- IAM
- ELB
- Auto Scaling
- VPC
- CloudFormation (optional)

Verify IAM Credentials

```
aws sts get-caller-identity
```

Example Output

```
{
    "UserId": "AIDA....",
    "Account": "123456789012",
    "Arn": "arn:aws:iam::123456789012:user/admin"
}
```

If this command works successfully, AWS CLI is configured correctly.

---

3. Amazon S3 Bucket

What is Amazon S3?

Amazon S3 (Simple Storage Service) is AWS's object storage service.

KOPS uses an S3 bucket to store the Kubernetes cluster configuration.

This bucket is called the **State Store**.

Think of it as the "database" of KOPS.

Why Does KOPS Need an S3 Bucket?

Every Kubernetes cluster contains:

- Certificates
- Secrets
- ETCD configuration
- Instance Groups
- Networking configuration
- Cluster YAML
- Add-on configuration

Instead of storing these locally, KOPS stores them inside S3.

Whenever you run:

```
kops edit cluster
```

KOPS reads the configuration from the S3 bucket.

Whenever you update the cluster:

```
kops update cluster --yes
```

KOPS writes the updated configuration back to the S3 bucket.

Create S3 Bucket

Example bucket name:

```
kops-demopurpose.k8s.local
```

Using AWS CLI

```
aws s3api create-bucket \
--bucket kops-demopurpose.k8s.local \
--region us-east-1
```

> **Note:** Bucket names must be globally unique across all AWS accounts.

Verify Bucket

```
aws s3 ls
```

Expected Output

2026-07-26 kops-demopurpose.k8s.local

Configure State Store

export KOPS_STATE_STORE=s3://kops-demopurpose.k8s.local

Verify

```
echo $KOPS_STATE_STORE
```

Output

```
s3://kops-demopurpose.k8s.local
```

---

4. Route53 Hosted Zone

What is DNS?

DNS (Domain Name System) converts domain names into IP addresses.

Example:

```
www.google.com
```

is translated into an IP address.

Why Does KOPS Need Route53?

Kubernetes API Server needs a permanent DNS name.

Instead of:

```
44.215.15.100
```

KOPS creates:

```
api.kops-demopurpose.k8s.local
```

This is managed by Route53.

Hosted Zone

A Hosted Zone stores DNS records.

Example:

```
kops-demopurpose.k8s.local
```

Inside this hosted zone, KOPS automatically creates DNS records.

Create Hosted Zone

AWS Console

```
Route53

↓

Hosted Zones

↓

Create Hosted Zone

↓

Domain Name

↓

kops-demopurpose.k8s.local

↓

Public Hosted Zone

↓

Create
```

Verify Hosted Zone

AWS Console

```
Route53

↓

Hosted Zones

↓

kops-demopurpose.k8s.local
```

You should see NS and SOA records.

---

5. EC2 Key Pair

What is a Key Pair?

A Key Pair allows secure SSH access to EC2 instances.

Without a key pair, you cannot log in to the Master or Worker Nodes.

Create Key Pair

AWS Console

```
EC2

↓

Key Pairs

↓
```

```
Create Key Pair

↓

Name

↓

kops-key

↓

Download PEM file
```

Store the `.pem` file securely.

Verify Key Pair

```
ls ~/.ssh/
```

Example

```
kops-key.pem
```

---

6. VPC

A Virtual Private Cloud is a logically isolated network within AWS.

KOPS creates resources inside a VPC.

The VPC provides:

- IP addresses
- Routing

- Internet access
- Security
- Isolation

VPC Components

```
VPC
│
├── Public Subnet
├── Private Subnet
├── Route Table
├── Internet Gateway
├── NAT Gateway
└── Security Groups
```

---

7. Security Groups

Security Groups act as firewalls.

They control traffic to EC2 instances.

Typical rules:

Port	Purpose
22	SSH
443	Kubernetes API
80	HTTP
6443	API Server
30000-32767	NodePort Services

---

8. Elastic Load Balancer

KOPS automatically creates an Elastic Load Balancer for the Kubernetes API Server.

Responsibilities:

- Receives kubectl requests
- Balances API traffic
- Provides a single endpoint
- Supports High Availability

---

9. AWS Region

Choose a region close to your users.

Example:

```
us-east-1
```

Other examples:

```
us-west-2
ap-south-1
eu-west-1
```

---

10. Availability Zone

Each AWS Region contains multiple Availability Zones.

Example:

```
us-east-1a

us-east-1b
```

```
us-east-1c
```

KOPS deploys nodes into these Availability Zones.

Production clusters often span multiple AZs for high availability.

AWS Infrastructure Checklist

Before creating a KOPS cluster, verify:

Item	Status
AWS Account	✅
Billing Enabled	✅
IAM User Created	✅
Access Keys Generated	✅
AWS CLI Installed	✅
kubectl Installed	✅
KOPS Installed	✅
S3 Bucket Created	✅
Route53 Hosted Zone Created	✅
Key Pair Created	✅
Region Selected	✅

Chapter 5: Installing and Configuring KOPS Environment

Learning Objectives

This chapter will cover:

- Install kubectl
- Install KOPS
- Configure the PATH variable
- Verify installations
- Configure the KOPS State Store
- Test the environment before cluster creation

Introduction

Before creating a Kubernetes cluster using KOPS, the local machine or EC2 instance must have the required software installed.

KOPS itself does not contain Kubernetes binaries. It requires several supporting tools.

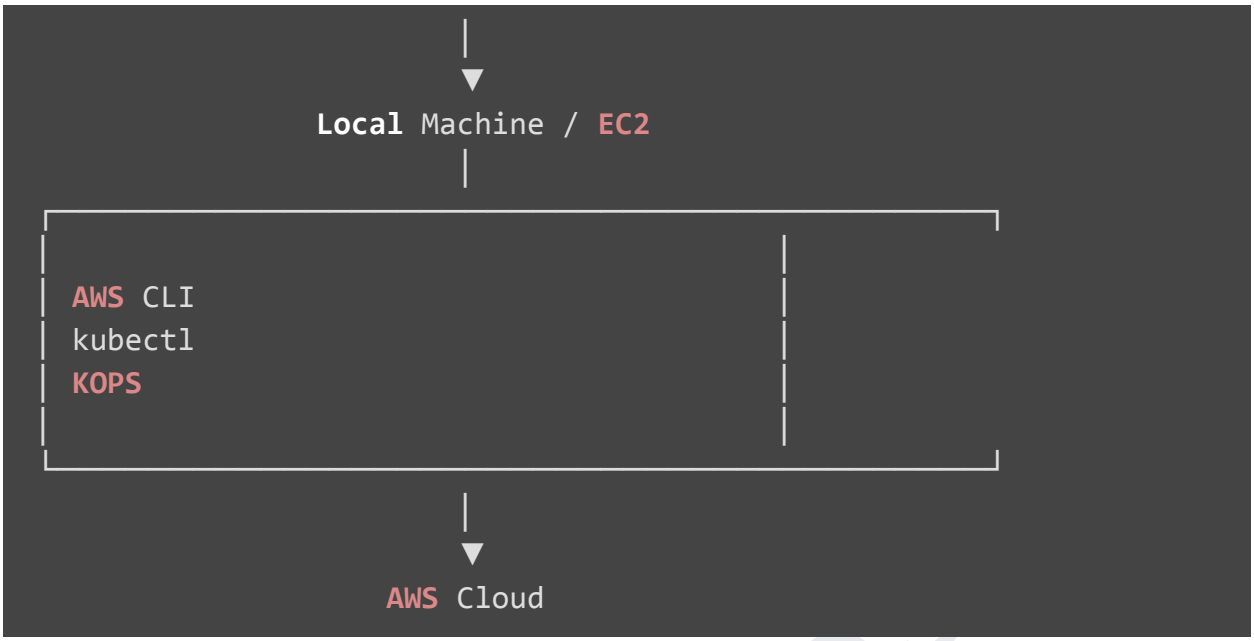
The tools required are:

- AWS CLI
- kubectl
- KOPS

These tools work together to create and manage the Kubernetes cluster.

Software Architecture

```
Administrator
```



Configure KOPS State Store

KOPS must know where the State Store is located.

Set the environment variable:

```
export KOPS_STATE_STORE=s3://kops-demopurpose.k8s.local
```

Replace `kops-demopurpose.k8s.local` with your actual S3 bucket name.

Explanation:

Command	Description
export	Creates an environment variable
KOPS_STATE_STORE	Variable read by KOPS
s3://	Indicates an Amazon S3 bucket
kops-demopurpose.k8s.local	Name of the State Store bucket

Verify the Environment Variable

```
echo $KOPS_STATE_STORE
```

Example Output

```
s3://kops-demopurpose.k8s.local
```

Chapter: Chapter 6 – Creating Your First Kubernetes Cluster Using KOPS

This chapter will cover:

- Understanding every parameter of the `kops create cluster` command
- Selecting regions and availability zones
- Choosing AMIs and instance types
- Creating the cluster configuration
- Updating the cluster to provision AWS resources
- Validating the cluster
- Verifying node status
- Understanding what KOPS creates internally during cluster provisioning

---

Introduction

After installing and configuring all required tools, the next step is to create a Kubernetes cluster.

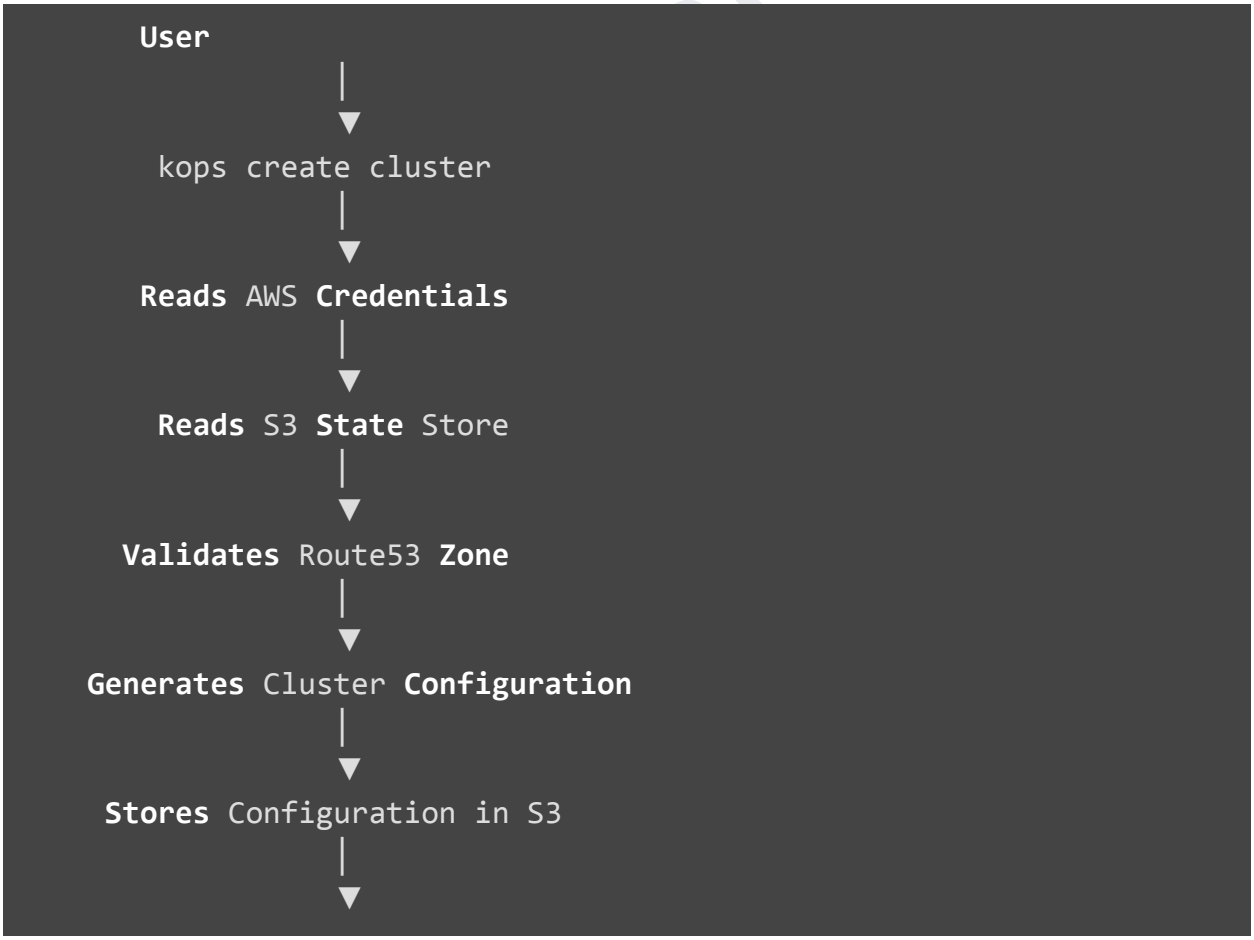
Unlike Minikube, which creates a local cluster on a single machine, KOPS creates a real Kubernetes cluster in AWS.

When the cluster is created, KOPS automatically provisions AWS resources such as:

- EC2 Instances
- IAM Roles
- Security Groups
- Elastic Load Balancer
- EBS Volumes
- Auto Scaling Groups
- Route53 DNS Records
- Kubernetes Control Plane
- Worker Nodes

This automation makes KOPS one of the simplest ways to deploy production-grade Kubernetes clusters on AWS.

Cluster Creation Workflow



```
kops update cluster --yes
        |
        ▼
Creates AWS Infrastructure
        |
        ▼
Installs Kubernetes Components
        |
        ▼
Worker Nodes Join Cluster
        |
        ▼
Cluster Becomes Available
```

Step 1: Configure the State Store

Step 2: Create the Cluster Configuration

The following command creates the Kubernetes cluster configuration. It does **not** create AWS resources yet.

```
kops create cluster \
--name kops-demopurpose.k8s.local \
--zones us-east-1a \
--control-plane-image ami-004f790b835b26145 \
--control-plane-count=1 \
--control-plane-size=t2.medium \
--image ami-004f790b835b26145 \
--node-count=2 \
--node-size=t2.medium
```

Understanding Every Parameter

1. Cluster Name

```
--name kops-demopurpose.k8s.local
```

This is the unique name of the Kubernetes cluster.

KOPS also uses this name to create DNS records in Route53.

Example:

```
api.kops-demopurpose.k8s.local
```

---

2. Availability Zone

```
--zones us-east-1a
```

This specifies where the EC2 instances will be created.

Example:

```
Region : us-east-1

Availability Zone : us-east-1a
```

Production clusters usually use multiple Availability Zones.

Example:

```
--zones us-east-1a,us-east-1b,us-east-1c
```

---

3. Control Plane Image

```
--control-plane-image ami-004f790b835b26145
```

Specifies the AMI used for the Control Plane EC2 instance.

The AMI determines:

- Operating System
- Installed packages
- Kernel version

---

4. Number of Control Plane Nodes

```
--control-plane-count=1
```

Creates one Control Plane node.

Development Cluster:

```
1 Control Plane
```

Production Cluster:

```
3 Control Plane Nodes
```

This provides High Availability.

---

5. Control Plane Instance Type

```
--control-plane-size=t2.medium
```

Determines the EC2 instance type.

Example:

Instance	vCPU	RAM
t2.micro	1	1 GB
t2.small	1	2 GB

t2.medium	2	4 GB
t3.medium	2	4 GB

---

6. Worker Node Image

```
--image ami-004f790b835b26145
```

Specifies the operating system for worker nodes.

---

7. Number of Worker Nodes

```
--node-count=2
```

Creates two worker nodes.

Architecture:

```
Master

↓

Worker-1

↓

Worker-2
```

---

8. Worker Instance Type

```
--node-size=t2.medium
```

Specifies the EC2 instance type for worker nodes.

What Happens Internally?

After running `kops create cluster`, KOPS:

1. Reads AWS credentials.
2. Connects to the S3 State Store.
3. Verifies the Route53 Hosted Zone.
4. Generates Kubernetes certificates.
5. Creates cluster configuration files.
6. Stores configuration in the S3 bucket.

Important: At this stage, **no EC2 instances are created**.

Step 3: Apply the Configuration

Now instruct KOPS to create the AWS infrastructure.

```
kops update cluster --name kops-demopurpose.k8s.local --yes --admin
```

Command Explanation

Parameter	Description
update cluster	Applies the stored configuration
--name	Cluster name
--yes	Confirms execution
--admin	Generates administrator credentials

What Happens During `kops update cluster`?

KOPS performs the following actions:

- Creates IAM Roles
- Creates Security Groups
- Creates Launch Templates
- Creates Auto Scaling Groups
- Launches EC2 instances
- Attaches EBS volumes
- Configures networking
- Installs Kubernetes
- Generates kubeconfig
- Configures certificates
- Registers worker nodes

AWS Resources Created

Resource	Purpose
EC2 Instances	Control Plane and Worker Nodes
IAM Roles	Permissions
Security Groups	Firewall
Elastic Load Balancer	Kubernetes API
Auto Scaling Groups	Node management
EBS Volumes	Persistent storage
Route53 Records	DNS
Launch Templates	EC2 configuration

Step 4: Validate the Cluster

Cluster creation takes several minutes.

Validate the cluster:

```
kops validate cluster --wait 10m
```

Explanation

Parameter	Description
validate cluster	Checks cluster health
--wait 10m	Waits up to 10 minutes

Expected output:

Your cluster kops-demopurpose.k8s.local is ready

Step 5: Verify the Nodes

List all Kubernetes nodes:

```
kubectl get nodes
```

Expected output:

NAME STATUS ROLES AGE
master-us-east-1a Ready control-plane 10m
ip-172-20-32-15 Ready node 8m
ip-172-20-45-20 Ready node 8m

Display Node Labels

```
kubectl get nodes --show-labels
```

This command displays labels such as:

- Topology
- Instance Type
- Region
- Zone
- Architecture

Cluster Information

```
kubectl cluster-info
```

Example:

Kubernetes control plane is running at https://api.kops-demopurpose.k8s.local
CoreDNS is running at ...

Verify Kubernetes Version

```
kubectl version
```

Displays:

- Client Version
- Server Version

Chapter 7: Exploring the Kubernetes Cluster

Learning Objectives

this chapter will cover :

- Understand what happens after the Kubernetes cluster is created.
- Verify the health of the Kubernetes cluster.
- Explore Kubernetes Nodes.
- Understand Namespaces.
- View System Pods.
- Explore Services.
- Understand Deployments.
- Learn basic kubectl commands.
- Understand the Kubernetes control plane components.

Introduction

Your Kubernetes cluster has been successfully created using KOPS.

However, creating the cluster is only the beginning.

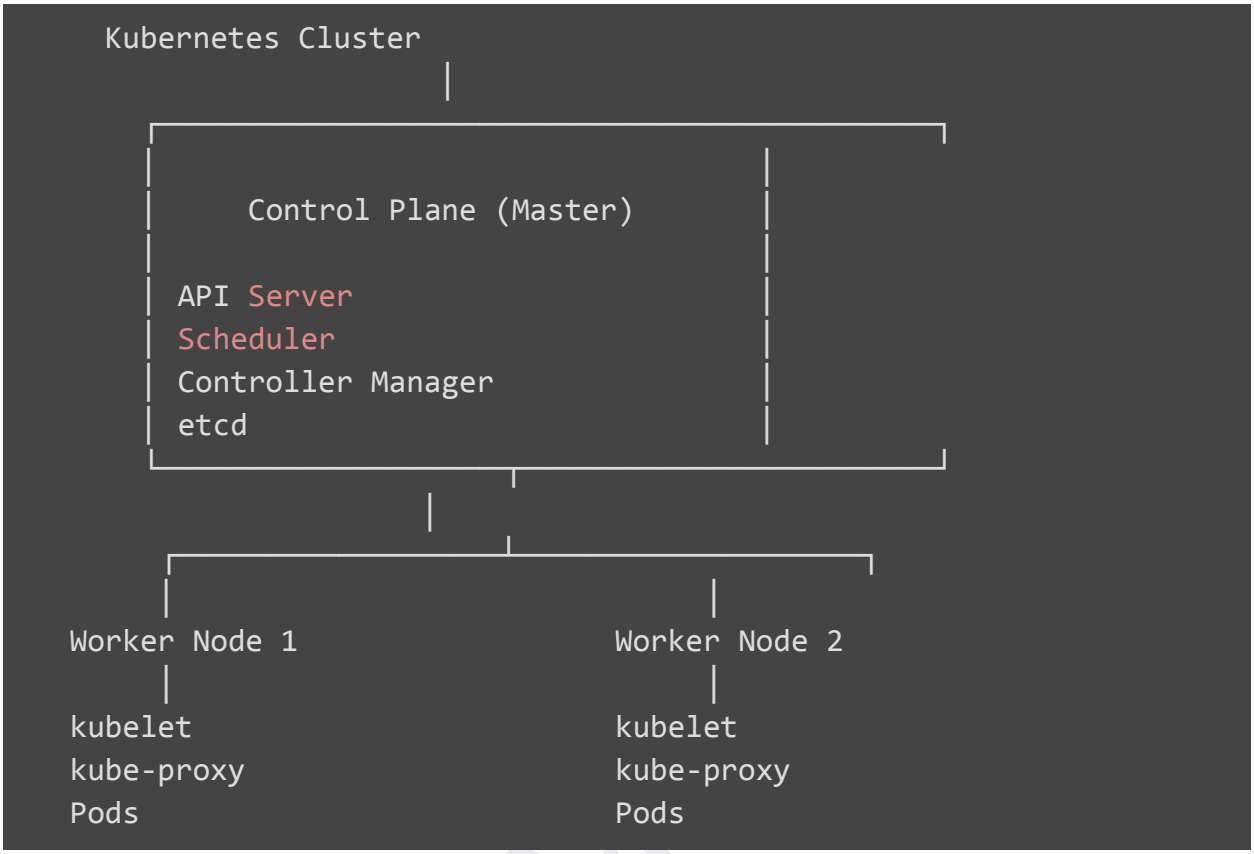
Before deploying any application, a Kubernetes administrator should always verify the health of the cluster.

This chapter focuses on understanding the cluster rather than deploying applications.

The commands covered in this chapter are essential for every Kubernetes administrator and are frequently used in production environments.

Kubernetes Cluster Overview

Once the cluster is ready, Kubernetes automatically creates several resources.



Step 1: Verify Nodes

Command

```
kubectl get nodes
```

Example Output

```
NAME                    STATUS   ROLES           AGE   VERSION
master-us-east-1a       Ready    control-plane   15m   v1.34.x
ip-172-20-40-120        Ready    node            13m   v1.34.x
ip-172-20-51-180        Ready    node            13m   v1.34.x
```

Explanation

This command displays every node registered in the Kubernetes cluster.

Each row represents one server participating in the cluster.

Understanding Each Column

NAME

The hostname of the node.

Example

```
master-us-east-1a
```

STATUS

Indicates whether Kubernetes can communicate with the node.

Possible values

Status	Meaning
Ready	Healthy
NotReady	Problem detected
Unknown	Communication lost

ROLES

Specifies the purpose of the node.

Example

```
control-plane
```

means Master Node.

Example

```
node
```

means Worker Node.

AGE

Shows how long the node has existed.

VERSION

Displays the Kubernetes version running on the node.

Display Node Labels

Command

```
kubectl get nodes --show-labels
```

Example Output

```
master-us-east-1a Ready control-plane beta.kubernetes.io/os=linux
```

What are Labels?

Labels are key-value pairs attached to Kubernetes resources.

Example

```
topology.kubernetes.io/zone=us-east-1a
```

Labels help Kubernetes:

- Schedule Pods
- Filter Resources
- Organize Nodes

Step 2: View Cluster Information

Command

```
kubectl cluster-info
```

Example Output

Kubernetes control plane is running at

```
https://api.kops-demopurpose.k8s.local
```

What does this command show?

It displays

- Kubernetes API Server URL
- CoreDNS URL
- Cluster endpoints

Step 3: Kubernetes Version

Command

```
kubectl version
```

Example Output

```
Client Version

Server Version
```

Client Version

Version of kubectl installed on your machine.

Server Version

Version of Kubernetes running inside AWS.

Step 4: View Namespaces

Command

```
kubectl get namespaces
```

Shortcut

```
kubectl get ns
```

Example Output

```
default

kube-system

kube-public

kube-node-lease
```

What is a Namespace?

A Namespace is a logical partition inside a Kubernetes cluster.

It allows multiple teams or applications to share the same cluster while remaining isolated.

Think of namespaces like folders in a computer.

Each folder stores different files.

Similarly,

Namespaces store different Kubernetes resources.

Default Namespaces

1. default

The default namespace for user applications.

If no namespace is specified, Kubernetes uses the default namespace.

---

2. kube-system

Contains Kubernetes system components.

Example

- CoreDNS
- kube-proxy
- Metrics Server
- Network Plugins

Never delete this namespace.

---

3. kube-public

Stores publicly accessible cluster information.

---

4. kube-node-lease

Used internally by Kubernetes to monitor node health.

Step 5: View All Pods

Command

```
kubectl get pods -A
```

Example Output

NAMESPACE NAME

kube-system coredns

kube-system kube-proxy

kube-system aws-node

What is a Pod?

A Pod is the smallest deployable unit in Kubernetes.

Every application runs inside Pods.

A Pod contains:

- One or more containers
- Network

- Storage

Understanding Pod Status

Possible values

Status	Meaning
Running	Healthy
Pending	Waiting
Completed	Finished
CrashLoopBackOff	Application crashing
ImagePullBackOff	Image download failed
Error	Pod failed

Describe a Pod

Command

```
kubectl describe pod <pod-name> -n kube-system
```

Example

```
kubectl describe pod coredns-xxxxxxxx -n kube-system
```

Why use Describe?

Shows

- Events

- Image Name
- IP Address
- Mounted Volumes
- Environment Variables
- Restart Count

Step 6: View Services

Command

```
kubectl get svc -A
```

Example Output

```
NAMESPACE     NAME

default       kubernetes

kube-system   kube-dns
```

What is a Service?

A Service provides a stable network endpoint for Pods.

Pods are temporary.

If a Pod is deleted,

its IP address changes.

A Service provides a permanent IP and DNS name.

Types of Services

Service Type	Usage
ClusterIP	Internal communication
NodePort	External access using Node IP
LoadBalancer	Cloud Load Balancer
ExternalName	Maps to external DNS

Step 7: View Deployments

Command

```
kubectl get deployments -A
```

Example

```
NAMESPACE

NAME

READY

AVAILABLE
```

What is a Deployment?

A Deployment manages Pods.

Instead of creating Pods directly,

we create Deployments.

Deployment automatically:

- Creates Pods

- Creates ReplicaSets
- Restarts failed Pods
- Performs Rolling Updates
- Supports Rollback

Deployment Architecture

```
Deployment
      |
      ▼
ReplicaSet
      |
      ▼
Pods
```

Difference Between Pod and Deployment

Pod	Deployment
Runs containers	Manages Pods
Manual recovery	Automatic recovery
No scaling	Supports scaling
No rollback	Supports rollback
Not recommended	Recommended

View All Resources

Command

```
kubectl get all
```

Example Output

```
Pods

Services

Deployments

ReplicaSets
```

Common Commands Summary

Command	Purpose
kubectl get nodes	View nodes
kubectl cluster-info	View cluster information
kubectl version	View versions
kubectl get ns	List namespaces
kubectl get pods -A	List all pods
kubectl describe pod	Detailed pod information
kubectl get svc -A	List services
kubectl get deployments -A	List deployments
kubectl get all	View common resources

Chapter 8 – Deploying Your First Application in Kubernetes (NGINX)

Learning Objectives

this chapter will cover:

- Understand what an application deployment is in Kubernetes.
- Learn why Deployments are preferred over Pods.
- Deploy an NGINX web server.
- Verify the Deployment.
- Understand ReplicaSets.
- Understand the Pod lifecycle.
- Inspect Pods and troubleshoot issues.
- Learn commonly used `kubectl` commands for deployments.

Introduction

Now that our Kubernetes cluster is running successfully, it's time to deploy our first application.

In this chapter, we will deploy the **NGINX Web Server**.

NGINX is a lightweight, high-performance web server that is commonly used to:

- Host websites
- Reverse Proxy
- Load Balancer
- API Gateway
- Static Content Server

In Kubernetes, we deploy applications using a **Deployment** object.

Why Do We Use Deployments?

A common question is:

Why not create Pods directly?

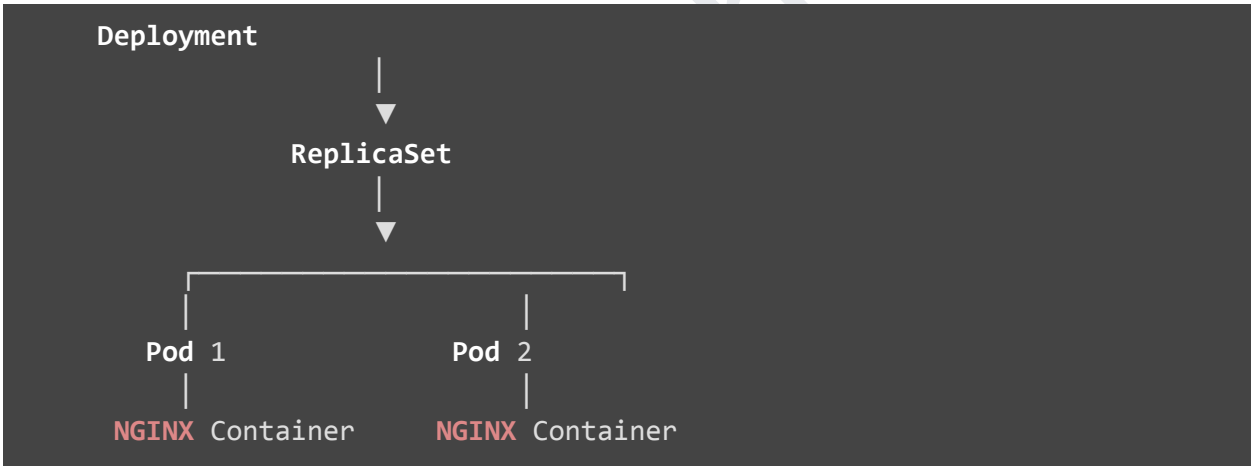
Pods are temporary resources. If a Pod crashes or the node hosting it fails, Kubernetes will not recreate it automatically unless it is managed by a controller.

A Deployment provides:

- Automatic Pod creation
- Automatic Pod recovery
- Rolling updates
- Rollbacks
- Scaling
- Self-healing

Therefore, in real-world projects, applications are almost always deployed using Deployments rather than standalone Pods.

Kubernetes Deployment Architecture



Understanding the Components

Deployment

A Deployment manages the entire application lifecycle.

Responsibilities:

- Creates ReplicaSets

- Creates Pods
- Replaces failed Pods
- Supports updates
- Supports rollback
- Supports scaling

ReplicaSet

A ReplicaSet ensures the required number of Pods are always running.

Example:

Desired replicas = **3**

Current replicas = **2**

ReplicaSet automatically creates one more Pod.

Pod

A Pod is the smallest deployable object in Kubernetes.

Each Pod contains:

- One or more Containers
- Storage
- Network
- Metadata

Container

A Container is where the actual application runs.

Example:

```
Pod
 └── NGINX Container
```

Step 1: Create an NGINX Deployment

Command

```
kubectl create deployment nginx --image=nginx
```

Command Breakdown

Part	Description
kubectl	Kubernetes CLI
create	Create a resource
deployment	Resource type
nginx	Deployment name
--image=nginx	Docker image to use

What Happens Internally?

When you execute the command:

```
kubectl create deployment nginx --image=nginx
```

Kubernetes performs the following sequence:

1. `kubectl` sends a request to the Kubernetes API Server.
2. The API Server validates the request.
3. A Deployment object is created.
4. The Deployment creates a ReplicaSet.
5. The ReplicaSet creates one Pod.
6. The Scheduler selects a Worker Node.
7. The Pod is assigned to the selected Worker Node.
8. The kubelet instructs the container runtime to pull the NGINX image.
9. The container starts.
10. The Pod status becomes **Running**.

Workflow Diagram

```
kubectl
    |
    ▼
API Server
    |
    ▼
Deployment
    |
    ▼
ReplicaSet
    |
    ▼
Scheduler
    |
    ▼
Worker Node
    |
    ▼
Pod
    |
    ▼
NGINX Container
```

Step 2: Verify the Deployment

```
kubectl get deployment
```

Example Output

```
NAME      READY   UP-TO-DATE   AVAILABLE   AGE
nginx     1/1     1            1           25s
```

Output Explanation

READY

Current Pods / Desired Pods

```
1/1
```

means one Pod is running successfully.

UP-TO-DATE

Shows whether the latest Deployment specification has been applied.

AVAILABLE

Number of Pods available to serve traffic.

AGE

Time since the Deployment was created.

Step 3: View ReplicaSets

```
kubectl get replicasets
```

Shortcut:

```
kubectl get rs
```

Example Output

```
NAME                DESIRED   CURRENT   READY
nginx-7c9fbd9df     1         1         1
```

What is a ReplicaSet?

A ReplicaSet maintains the desired number of Pods.

If one Pod is deleted:

```
Desired Pods : 1

Current Pods : 0
```

ReplicaSet immediately creates another Pod.

Step 4: View Pods

```
kubectl get pods
```

Example Output

```
NAME                      READY   STATUS    RESTARTS   AGE
nginx-7c9fbd9df-q7hwd     1/1     Running   0          2m
```

Output Explanation

READY

Container status inside the Pod.

```
1/1
```

means one container is healthy.

STATUS

Shows the current Pod state.

Possible values:

Status	Meaning
Running	Healthy
Pending	Waiting for scheduling
Completed	Finished successfully
Error	Failed
CrashLoopBackOff	Repeated crashes
ImagePullBackOff	Image download failed

RESTARTS

Number of times the container has restarted.

AGE

Time since the Pod was created.

Step 5: Describe the Pod

```
kubectl describe pod <pod-name>
```

Example

```
kubectl describe pod nginx-7c9fbd9df-q7hwd
```

Information Displayed

- Node name
- IP address
- Labels
- Docker image
- Environment variables
- Mounted volumes
- Events
- Container status
- Restart count

Step 6: View Pod Logs

```
kubectl logs <pod-name>
```

Example

```
kubectl logs nginx-7c9fbd9df-q7hwd
```

Why Are Logs Important?

Logs help identify:

- Application errors
- Startup failures
- Configuration problems
- Runtime issues

Step 7: Watch the Deployment

```
kubectl get pods -w
```

The `-w` flag stands for **watch**.

This command continuously displays updates as Pods are created, deleted, or restarted.

Understanding the Pod Lifecycle

Every Pod goes through several stages during its lifetime.

```
Pending
   |
   ▼
ContainerCreating
   |
   ▼
Running
   |
   ▼
```

```
Succeeded / Failed
```

Pending

The Pod has been accepted by Kubernetes but is waiting for scheduling or resources.

ContainerCreating

The container runtime is pulling the image and preparing the container.

Running

The application is running successfully.

Succeeded

The Pod completed its task successfully (common for Jobs).

Failed

The Pod terminated due to an error.

Common Deployment Commands

List Deployments

```
kubectl get deployment
```

List ReplicaSets

```
kubectl get rs
```

List Pods

```
kubectl get pods
```

Describe Deployment

```
kubectl describe deployment nginx
```

Describe Pod

```
kubectl describe pod <pod-name>
```

View Logs

```
kubectl logs <pod-name>
```

Watch Pods

```
kubectl get pods -w
```

Chapter 10: Exposing Applications Using Kubernetes Services

Learning Objectives

this chapter will cover:

- Understand why Pods cannot be accessed directly.
- Learn what a Kubernetes Service is.
- Understand the different types of Kubernetes Services.
- Create a NodePort Service.
- Access an application from a web browser.
- Understand Service networking.
- Verify and troubleshoot Services.

Introduction

In the previous chapter, we successfully deployed an NGINX application.

Although the application is running inside a Pod, it cannot yet be accessed from outside the Kubernetes cluster.

Why?

Because Pods are temporary resources.

- Pod IP addresses are dynamic.
- Pods may be recreated at any time.
- When a Pod is recreated, it receives a new IP address.

If users directly access the Pod IP, the application becomes unavailable whenever the Pod is recreated.

To solve this problem, Kubernetes introduces the concept of **Services**.

A Service provides a stable network endpoint that always forwards requests to the correct Pod(s), even if Pods are recreated.

Why Can't We Access a Pod Directly?

Suppose we deploy an NGINX Pod.

```
      Worker Node

          +-------------------+
          |                   |
          |     NGINX Pod     |
          |                   |
          | IP: 10.244.1.15   |
          +-------------------+
```

If the Pod crashes:

```
Old Pod → Deleted

↓

New Pod Created

↓

New IP = 10.244.2.10
```

The application IP changes.

Users cannot rely on changing Pod IPs.

Solution: Kubernetes Service

A Kubernetes Service provides a permanent IP address and DNS name.

Instead of users connecting directly to Pods:

```
User

↓

Pod IP
```

Users connect to the Service.

```
User

↓

Service

↓

Pod
```

Even if Pods change, the Service continues routing traffic correctly.

Kubernetes Service Architecture

```
            User
              |
              ▼
        Kubernetes Service
```

The Service automatically load-balances requests across all healthy Pods.

What is a Kubernetes Service?

A Kubernetes Service is a networking abstraction that provides:

- A stable IP address
- A stable DNS name
- Load balancing
- Service discovery
- Communication between applications

Services use **selectors** to identify which Pods should receive traffic.

Types of Kubernetes Services

Kubernetes supports four main Service types.

Service Type	Purpose
ClusterIP	Internal communication within the cluster
NodePort	Exposes the application using the Worker Node IP
LoadBalancer	Creates a cloud load balancer (AWS ELB, Azure LB, etc.)
ExternalName	Maps the Service to an external DNS name

---

1. ClusterIP (Default)

ClusterIP creates an internal virtual IP.

Only Pods inside the cluster can access it.

Example:

```
Pod A

↓

ClusterIP Service

↓

Pod B
```

Users outside the cluster cannot access this Service.

---

2. NodePort

NodePort exposes the application on every Worker Node using a high port number.

Example:

```
Worker Node IP

↓

192.168.1.10:30080
```

Anyone with the Worker Node IP and NodePort can access the application.

This is commonly used in learning and testing environments.

---

3. LoadBalancer

This Service type creates a cloud load balancer automatically.

In AWS, Kubernetes provisions an Elastic Load Balancer (ELB).

```
Internet

↓

AWS Load Balancer

↓

Service

↓

Pods
```

Recommended for production applications.

---

4. ExternalName

Instead of forwarding traffic to Pods, this Service maps to an external DNS name.

Example:

```
database-service

↓

mysql.example.com
```

Useful when applications need to access external services.

Creating a NodePort Service

To expose the NGINX Deployment externally:

```
kubectl expose deployment nginx --type=NodePort --port=80
```

Command Breakdown

Part	Description
kubectl	Kubernetes CLI
expose	Create a Service
deployment	Resource type
nginx	Deployment name
--type=NodePort	Service type
--port=80	Application port

What Happens Internally?

When this command is executed:

1. Kubernetes creates a Service object.
2. The Service receives a ClusterIP.
3. Kubernetes assigns a NodePort (30000–32767).
4. The Service identifies Pods using labels.
5. Traffic arriving at the NodePort is forwarded to one of the Pods.

Verify the Service

Run:

```
kubectl get svc
```

Example Output:

```
NAME         TYPE       CLUSTER-IP      EXTERNAL-IP   PORT(S)        AGE
kubernetes   ClusterIP  100.64.0.1      <none>        443/TCP        1h
nginx        NodePort   100.64.12.25    <none>        80:31245/TCP   2m
```

Understanding the Output

NAME

The Service name.

Example:

```
nginx
```

TYPE

Displays the Service type.

Example:

```
NodePort
```

CLUSTER-IP

Internal IP assigned to the Service.

Pods communicate using this IP.

PORT(S)

```
80:31245/TCP
```

Meaning:

- Container Port = 80
- NodePort = 31245

Accessing the Application

Find the Worker Node Public IP.

Example:

```
54.221.100.25
```

NodePort:

```
31245
```

Open your browser:

```
http://54.221.100.25:31245
```

Expected Output:

```
Welcome to nginx!
```

Congratulations! 🎉

Your Kubernetes application is now accessible from outside the cluster.

Describe the Service

kubectl describe svc nginx

This command displays:

- Labels
- Selectors

- Endpoints
- Cluster IP
- NodePort
- Events

How Does Kubernetes Know Which Pods to Send Traffic To?

Kubernetes uses **Labels** and **Selectors**.

Example Pod Label:

```
app=nginx
Service Selector:
app=nginx
```

The Service automatically discovers Pods with matching labels.

View Labels

```
kubectl get pods --show-labels
```

Example:

```
NAME                    LABELS

nginx-xxxxx             app=nginx
```

Service Traffic Flow

```
    Browser
```

Traffic is distributed across healthy Pods.

Common Commands

Create Service

```
kubectl expose deployment nginx --type=NodePort --port=80
```

View Services

```
kubectl get svc
```

Describe Service

```
kubectl describe svc nginx
```

Delete Service

```
kubectl delete svc nginx
```

View Endpoints

```
kubectl get endpoints
```

Chapter 11: Scaling Applications in Kubernetes

Learning Objectives

After completing this chapter, students will be able to:

- Understand what scaling means in Kubernetes.
- Learn the difference between Horizontal Scaling and Vertical Scaling.
- Scale an application using the `kubectl scale` command.
- Verify the number of replicas.
- Understand the role of ReplicaSets.
- Learn Kubernetes Self-Healing.
- Observe automatic Pod creation and deletion.

Introduction

Imagine you own an e-commerce website.

On a normal day, around **100 users** visit your website.

During a festival sale, suddenly **10,000 users** visit at the same time.

If only one application instance is running, it may become overloaded and stop responding.

To handle increasing traffic, we need **Scaling**.

Kubernetes makes scaling very simple and automatic.

What is Scaling?

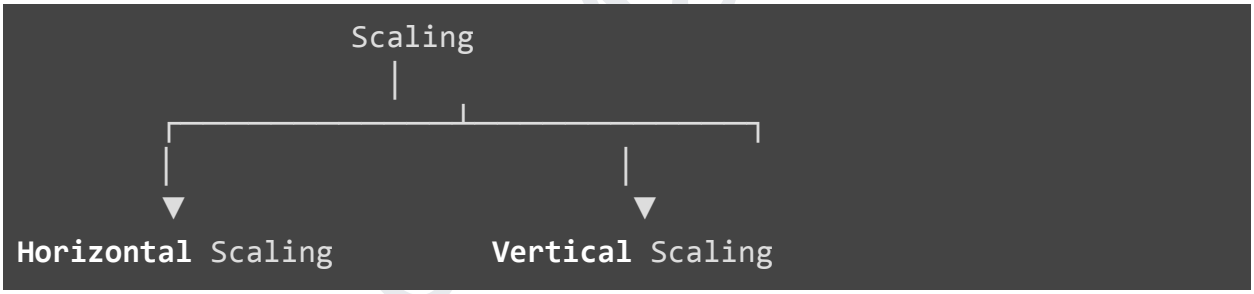
Scaling is the process of increasing or decreasing application resources based on workload.

Scaling helps applications:

- Handle more users
- Improve performance
- Improve availability
- Prevent downtime
- Reduce resource wastage

Types of Scaling

There are **two** types of scaling.



---

1. Horizontal Scaling

Horizontal Scaling means **adding more Pods**.

Example

Initially

```
↓

1 Pod
```

After Scaling

```
NGINX Deployment

↓

5 Pods
```

Traffic is distributed among all Pods.

Advantages of Horizontal Scaling

- High Availability
- Load Balancing
- Better Performance
- Zero Downtime
- Fault Tolerance

---

2. Vertical Scaling

Vertical Scaling means increasing the resources of an existing server.

Example

Before

```
2 CPU
```

```
4 GB RAM
```

After

```
8 CPU

16 GB RAM
```

Instead of adding more Pods, the existing Pod receives more CPU and Memory.

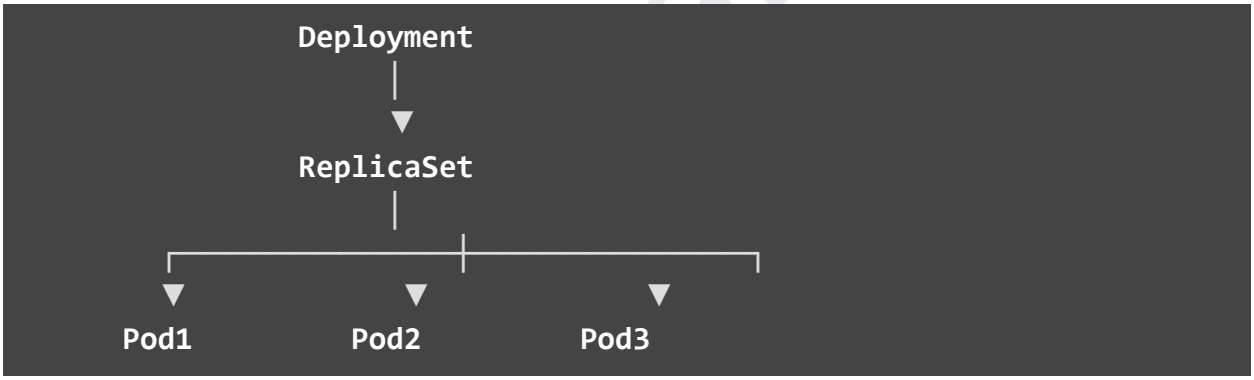
Horizontal vs Vertical Scaling

Horizontal Scaling	Vertical Scaling
Adds Pods	Adds CPU/RAM
Better Availability	Limited Availability
Supports Load Balancing	No Load Balancing
Kubernetes Preferred	Used less frequently
Easy	Limited by hardware

Kubernetes Scaling Architecture

Before Scaling

After Scaling



Step 1: View Current Deployment

Before scaling, check the current deployment.

```
kubectl get deployment
```

Example Output

```
NAME      READY   UP-TO-DATE   AVAILABLE   AGE
nginx     1/1     1            1           15m
```

The Deployment currently has **one replica**.

Step 2: Scale the Deployment

Increase the replicas to **5**.

```
kubectl scale deployment nginx --replicas=5
```

Command Explanation

Part	Description
kubectl	Kubernetes CLI
scale	Scale a resource
deployment	Resource type
nginx	Deployment name

--replicas=5	Desired number of Pods

What Happens Internally?

When the command is executed:

1. kubectl sends the request to the API Server.
2. Deployment updates the desired replica count.
3. ReplicaSet detects the change.
4. Scheduler selects Worker Nodes.
5. New Pods are created.
6. kubelet starts containers.
7. Pods become **Running**.
8. Service automatically starts sending traffic to the new Pods.

Scaling Workflow

```
kubectl scale

↓

API Server

↓

Deployment

↓

ReplicaSet
```

```
↓
Scheduler
↓
Worker Nodes
↓
5 Pods Running
```

Step 3: Verify the Deployment

```
kubectl get deployment
```

Example Output

```
NAME      READY   UP-TO-DATE   AVAILABLE   AGE
nginx     5/5     5            5           20m
```

Explanation:

READY

```
5/5
```

means all five Pods are healthy.

Step 4: Verify Pods

```
kubectl get pods
```

Example Output

```
NAME                          READY   STATUS
nginx-7c9fbd9df-abc12         1/1     Running
nginx-7c9fbd9df-def34         1/1     Running
nginx-7c9fbd9df-ghi56         1/1     Running
nginx-7c9fbd9df-jkl78         1/1     Running
nginx-7c9fbd9df-mno90         1/1     Running
Now five Pods are running.
```

Step 5: View ReplicaSet

```
kubectl get rs
```

Example

```
NAME              DESIRED CURRENT READY

nginx-xxxxx       5       5       5
```

ReplicaSet ensures that exactly **five Pods** remain running.

Scaling Down

Scaling is not limited to increasing Pods.

We can also reduce them.

Example

```
kubectl scale deployment nginx --replicas=2
```

Now Kubernetes deletes the extra Pods gracefully.

Verify Again

```
kubectl get pods
```

Output

```
Only two Pods remain Running.
```

Self-Healing in Kubernetes

One of Kubernetes' most powerful features is **Self-Healing**.

Suppose one Pod crashes.

Kubernetes automatically creates a replacement Pod.

Demonstration

First, list the Pods.

```
kubectl get pods
```

Example

```
nginx-7c9fbd9df-abc12

nginx-7c9fbd9df-def34
```

Delete one Pod manually.

```
kubectl delete pod nginx-7c9fbd9df-abc12
```

What Happens?

Immediately after deletion:

```
ReplicaSet

↓

Current Pods = 1

Desired Pods = 2

↓

Create New Pod

↓

Running
```

Within a few seconds, Kubernetes creates a new Pod automatically.

Verify:

```
kubectl get pods
```

Notice that a new Pod appears with a different name.

Why Does the Pod Name Change?

Pods are **immutable**.

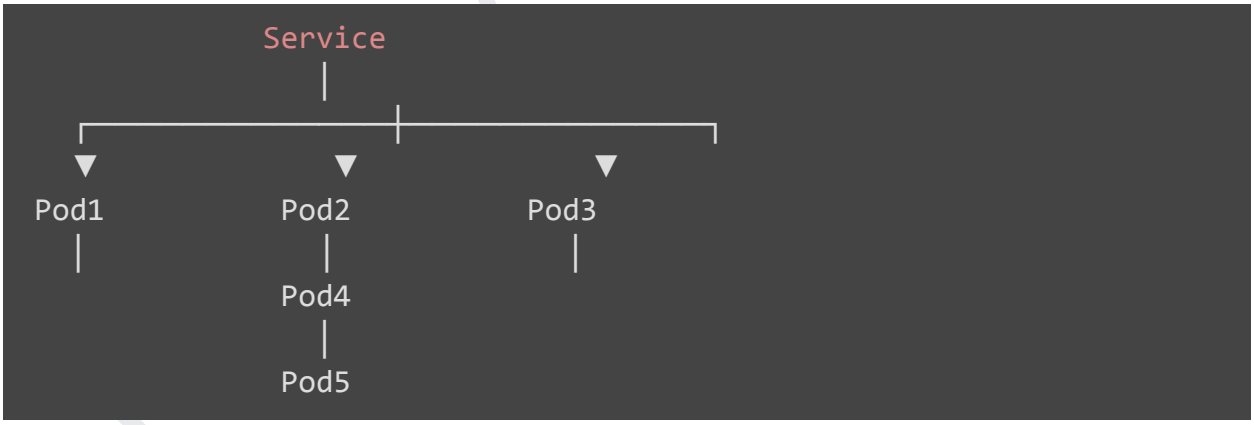
When a Pod is deleted, Kubernetes does not restart the same Pod.

Instead, it creates a brand-new Pod with:

- New name
- New UID
- New IP Address

Load Balancing During Scaling

Suppose five Pods are running.



The Service distributes requests among all healthy Pods.

This improves performance and reliability.

Watching Scaling in Real Time

Open a second terminal.

Run:

```
kubectl get pods -w
```

Now execute the scaling command in the first terminal.

```
kubectl scale deployment nginx --replicas=5
```

You will see Pods being created live.

Common Commands

View Deployment

```
kubectl get deployment
```

Scale Up

```
kubectl scale deployment nginx --replicas=5
```

Scale Down

```
kubectl scale deployment nginx --replicas=2
```

View Pods

```
kubectl get pods
```

View ReplicaSet

```
kubectl get rs
```

Delete Pod

```
kubectl delete pod <pod-name>
```

Watch Pods

```
kubectl get pods -w
```

Chapter 12: Rolling Updates and Rollbacks in Kubernetes

Learning Objectives

After completing this chapter, you will be able to:

- Understand what a Rolling Update is.
- Learn why Rolling Updates are used instead of traditional deployments.
- Update a running application with zero downtime.
- Monitor the rollout process.
- View rollout history.
- Perform a rollback to a previous version.
- Understand Kubernetes Deployment strategies.

Introduction

Suppose your company has an e-commerce application running Version 1.

Everything is working perfectly.

Now, the development team releases **Version 2** with:

- New features

- Bug fixes
- Performance improvements

How do you deploy the new version?

There are two approaches:

Traditional Deployment

```
Stop Version 1

↓

Deploy Version 2

↓

Start Application
```

Problem:

Users experience **downtime** because the application is unavailable during the upgrade.

Kubernetes Rolling Update

```
Version 1

↓

Replace One Pod

↓

Replace Another Pod

↓

Replace Remaining Pods
```

```
↓

Version 2
```

The application remains available throughout the update.

This is known as a **Rolling Update**.

What is a Rolling Update?

A Rolling Update is a deployment strategy where Kubernetes gradually replaces old Pods with new Pods.

Instead of deleting all Pods at once, Kubernetes updates them one by one (or in small batches).

Benefits:

- Zero downtime
- High availability
- Safer deployments
- Easy rollback if something goes wrong

Traditional Deployment vs Rolling Update

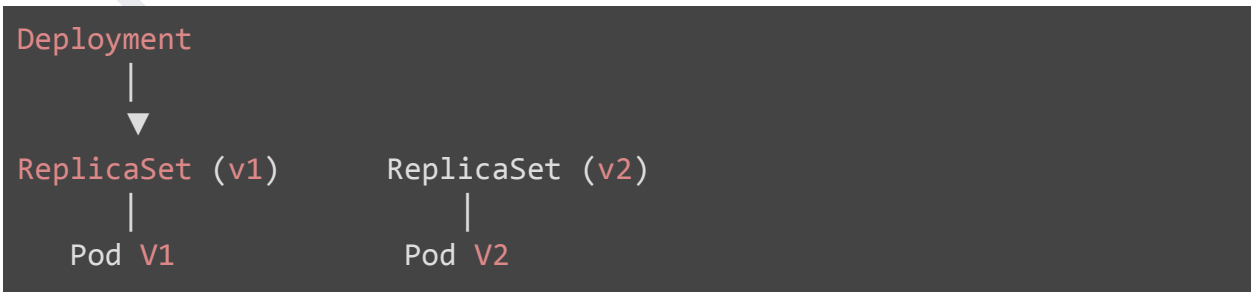
Traditional Deployment	Rolling Update
Stops application	Application remains available

Causes downtime	Zero or minimal downtime
High risk	Lower risk
Difficult recovery	Easy rollback
Not recommended	Recommended

Kubernetes Rolling Update Architecture

Before Update:

During Update:



```
      |                    |
   Pod V1              Pod V2
```

After Update:

```
Deployment
      |
      ▼
ReplicaSet (v2)
      |
  ┌───┴──────┐
  ▼          ▼
Pod V2    Pod V2
```

Step 1: Check the Current Deployment

```
kubectl get deployment
```

Example Output:

```
NAME      READY   UP-TO-DATE   AVAILABLE   AGE
nginx     2/2     2            2           20m
```

Step 2: Check the Current Image

```
kubectl describe deployment nginx
```

Look for:

```
Image:
nginx
```

This confirms the current container image.

Step 3: Update the Image

Suppose we want to update the NGINX image to the latest version.

```
kubectl set image deployment/nginx nginx=nginx:latest
```

Command Explanation

Part	Description
kubectl	Kubernetes CLI
set image	Update the container image
deployment/nginx	Deployment name
nginx=nginx:latest	Container name = New image

What Happens Internally?

When this command is executed:

1. Kubernetes updates the Deployment specification.
2. A new ReplicaSet is created.
3. One old Pod is terminated.
4. One new Pod is created with the updated image.
5. Health checks verify the new Pod.
6. The process repeats until all Pods are updated.
7. The old ReplicaSet is retained for rollback.

Rolling Update Workflow

```
kubectl set image
        |
        ▼
API Server
        |
        ▼
Deployment Updated
        |
        ▼
New ReplicaSet Created
        |
        ▼
Old Pods Deleted Gradually
        |
        ▼
New Pods Created
        |
        ▼
Update Completed
```

Step 4: Watch the Rolling Update

Open another terminal and run:

```
kubectl get pods -w
```

Example:

```
nginx-7f4d8d5b4-x2abc   Terminating

nginx-6bd4d75d7-m8xyz   Running
```

Notice:

- Old Pods terminate one at a time.
- New Pods start immediately.
- The application remains available.

Step 5: Check Rollout Status

```
kubectl rollout status deployment nginx
```

Example Output:

```
deployment "nginx" successfully rolled out
```

This command confirms whether the update completed successfully.

Step 6: View Rollout History

```
kubectl rollout history deployment nginx
```

Example Output:

```
REVISION    CHANGE-CAUSE

1

2
```

Each successful update creates a new revision.

Step 7: View Deployment Details

```
kubectl describe deployment nginx
```

This displays:

- Image version
- ReplicaSets
- Events
- Deployment conditions
- Strategy

What is a Revision?

Every time you update a Deployment, Kubernetes creates a new revision.

Example:

Revision	Image
Revision 1	nginx:1.27

Revision 2	nginx:latest
Revision 3	nginx:stable

If a deployment fails, Kubernetes can return to a previous revision.

What is Rollback?

Rollback means reverting the application to a previous working version.

Example:

```
Version 1 → Working

↓

Version 2 → Bug

↓

Rollback

↓

Version 1 Restored
```

Step 8: Roll Back the Deployment

```
kubectl rollout undo deployment nginx
```

Kubernetes restores the previous ReplicaSet automatically.

Verify the Rollback

```
kubectl rollout status deployment nginx
```

Expected Output:

```
deployment "nginx" successfully rolled out
```

View the Updated Pods

```
kubectl get pods
```

Pods are recreated using the previous image version.

Deployment Strategies

Kubernetes supports multiple deployment strategies.

Strategy	Description

RollingUpdate	Default strategy with zero/minimal downtime
Recreate	Deletes all old Pods before creating new ones

RollingUpdate (Default)

```
Pod V1

↓

Pod V2

↓

Pod V1

↓

Pod V2
```

Application stays available.

Recreate Strategy

```
Delete All Pods

↓

Create New Pods
```

This causes downtime and is used only when applications cannot run multiple versions simultaneously.

Common Commands

Check Deployment

```
kubectl get deployment
```

Update Image

```
kubectl set image deployment/nginx nginx=nginx:latest
```

Watch Pods

```
kubectl get pods -w
```

Check Rollout Status

```
kubectl rollout status deployment nginx
```

View Rollout History

```
kubectl rollout history deployment nginx
```

Rollback

```
kubectl rollout undo deployment nginx
```

Describe Deployment

```
kubectl describe deployment nginx
```

Chapter 13: Kubernetes YAML Manifests (Declarative Approach)

Learning Objectives

After completing this chapter, students will be able to:

- Understand what YAML is.
- Learn why Kubernetes uses YAML files.
- Understand Imperative vs Declarative approaches.
- Learn the structure of a Kubernetes YAML file.
- Create Deployments using YAML.
- Create Services using YAML.
- Apply, update, and delete Kubernetes resources using YAML.
- Follow YAML best practices.

Introduction

So far, we have created Kubernetes resources using commands like:

```
kubectl create deployment nginx --image=nginx
```

This is called the **Imperative Approach** because we directly instruct Kubernetes to perform an action.

However, in real-world DevOps projects, engineers rarely use imperative commands for production deployments.

Instead, they define the desired infrastructure in **YAML files** and let Kubernetes create or update resources accordingly.

This is known as the **Declarative Approach**.

It is the standard practice used in GitOps, CI/CD pipelines, and production environments.

What is YAML?

YAML stands for:

YAML Ain't Markup Language

It is a human-readable data serialization language used to define configuration.

Kubernetes uses YAML files to describe resources such as:

- Pods
- Deployments
- Services
- ConfigMaps
- Secrets
- Ingress
- Persistent Volumes
- Jobs
- CronJobs

Why Does Kubernetes Use YAML?

YAML offers several advantages:

- Easy to read
- Easy to write
- Version-controlled using Git
- Reusable
- Supports automation
- Ideal for Infrastructure as Code (IaC)

Imperative vs Declarative

Imperative Approach

You tell Kubernetes **how** to perform an action.

Example:

```
kubectl create deployment nginx --image=nginx
```

Characteristics:

- Quick
- Good for learning
- Good for testing
- Difficult to maintain large projects

Declarative Approach

You define the desired state in a YAML file.

Example:

```
kubectl apply -f deployment.yaml
```

Characteristics:

- Preferred in production
- Easy to manage
- Easy to update
- Easy to rollback
- Stored in Git repositories

Imperative vs Declarative Comparison

Imperative	Declarative
Uses commands	Uses YAML files
Manual	Automated
Difficult to track	Easy with Git
Not reusable	Highly reusable
Good for testing	Best for production

Kubernetes YAML Structure

Every Kubernetes YAML file contains four major sections.

```
apiVersion
kind
metadata
spec
```

Example structure:

```yaml
apiVersion: apps/v1

kind: Deployment

metadata:

spec:
```

---

1. apiVersion

Specifies which Kubernetes API version should process the resource.

Examples:

Resource	apiVersion
Deployment	apps/v1
Service	v1
Pod	v1
ConfigMap	v1
Secret	v1

Example:

```
apiVersion: apps/v1
```

---

2. kind

Specifies the Kubernetes object type.

Examples:

```
kind: Deployment
```

Other examples:

```
Pod

Service

ConfigMap

Secret

Ingress

Job
```

---

3. metadata

Contains information about the resource.

Example:

```yaml
metadata:
  name: nginx-deployment
```

Metadata may contain:

- Name
- Namespace
- Labels
- Annotations

Example:

```yaml
metadata:
  name: nginx-deployment
  labels:
    app: nginx
```

---

4. spec

The most important section.

It defines the desired state of the resource.

For a Deployment, it specifies:

- Number of replicas
- Labels
- Containers
- Images
- Ports

Complete Deployment YAML

Create a file named **deployment.yaml**

```yaml
apiVersion: apps/v1
kind: Deployment

metadata:
  name: nginx-deployment

spec:
  replicas: 3

  selector:
    matchLabels:
      app: nginx

  template:

    metadata:
      labels:
        app: nginx

    spec:

      containers:

      - name: nginx

        image: nginx

        ports:

        - containerPort: 80
```

Understanding Every Section

replicas

```
replicas: 3
```

Creates three Pods.

---

selector

```
selector:
  matchLabels:
    app: nginx
```

The Deployment uses this label to identify which Pods it manages.

---

template

The Pod template.

Everything inside this section describes how the Pods should be created.

---

containers

Defines one or more containers.

Example:

```
containers:

- name: nginx
```

---

image

Specifies the Docker image.

```
image: nginx
```

---

containerPort

```
containerPort: 80
```

Specifies that the container listens on port 80.

Apply the YAML File

Create the Deployment:

```
kubectl apply -f deployment.yaml
```

Expected Output:

```
deployment.apps/nginx-deployment created
```

Verify the Deployment

```
kubectl get deployment
```

Example:

```
NAME                READY

nginx-deployment    3/3
```

Verify Pods

```
kubectl get pods
```

Example:

NAME

```
nginx-deployment-xxxxx

nginx-deployment-yyyyy

nginx-deployment-zzzzz
```

Creating a Service Using YAML

Create a file named **service.yaml**

```
apiVersion: v1

kind: Service

metadata:
  name: nginx-service
```

```
spec:

  selector:
    app: nginx

  ports:

  - protocol: TCP

    port: 80

    targetPort: 80

    nodePort: 30080

  type: NodePort
```

Understanding the Service YAML

selector

```
selector:
  app: nginx
```

Matches Pods with the label:

```
app: nginx
```

---

port

The Service port.

```
port: 80
```

---

targetPort

The container port.

```
targetPort: 80
```

---

nodePort

The external port.

```
nodePort: 30080
```

NodePort range:

```
30000-32767
```

---

type

```
type: NodePort
```

Makes the application accessible from outside the cluster.

Apply the Service

```
kubectl apply -f service.yaml
```

Verify

```
kubectl get svc
```

Example:

```
NAME            TYPE

nginx-service   NodePort
```

Updating Resources

Suppose we want five Pods instead of three.

Edit:

```
replicas: 5
```

Apply again.

```
kubectl apply -f deployment.yaml
```

Kubernetes compares the current state with the desired state and updates only what has changed.

Deleting Resources

Delete the Deployment.

```
kubectl delete -f deployment.yaml
```

Delete the Service.

```
kubectl delete -f service.yaml
```

YAML Workflow

```
Write YAML File

↓

Save

↓

kubectl apply -f file.yaml

↓

API Server

↓

Resource Created

↓

Verify
```

YAML Best Practices

1. Use meaningful names

Good:

```
name: nginx-deployment
```

Bad:

```
name: demo
```

---

2. Use labels consistently

Example:

```
labels:
  app: nginx
```

---

3. Store YAML files in Git

Example:

```
Git Repository

↓

deployment.yaml

service.yaml

configmap.yaml
```

---

4. Avoid hardcoding environment-specific values

Use ConfigMaps and Secrets where appropriate.

---

5. Validate YAML before applying

```
kubectl apply --dry-run=client -f deployment.yaml
```

This checks the manifest syntax and client-side validation without creating resources.

Common Commands

Create

```
kubectl apply -f deployment.yaml
```

Update

```
kubectl apply -f deployment.yaml
```

View

```
kubectl get deployment
```

Delete

```
kubectl delete -f deployment.yaml
```

Describe

```
kubectl describe deployment nginx-deployment
```

Common Errors

YAML Parsing Error

Example:

```
error converting YAML to JSON
```

Cause:

- Incorrect indentation
- Missing colon (`:`)
- Invalid YAML syntax

Solution:

Check spacing carefully. YAML is indentation-sensitive.

Selector Does Not Match Labels

If the Deployment selector and Pod labels differ, Pods will not be managed correctly.

Correct example:

```
selector:
  matchLabels:
    app: nginx

template:
  metadata:
    labels:
      app: nginx
```

Service Has No Endpoints

Cause:

The Service selector does not match the Pod labels.

Verify:

```
kubectl get pods --show-labels
kubectl describe svc nginx-service
```

Practical Exercise

Task 1

```
Create deployment.yaml with 3 replicas.
```

Task 2

Apply the Deployment.

```
kubectl apply -f deployment.yaml
```

Task 3

Verify the Pods.

```
kubectl get pods
```

Task 4

Create `service.yaml`.

Task 5

Apply the Service.

```
kubectl apply -f service.yaml
```

Task 6

Verify the Service.

```
kubectl get svc
```

Task 7

Update the Deployment to **5 replicas**.

```
replicas: 5
```

Apply again:

```
kubectl apply -f deployment.yaml
```

Task 8

Delete the resources.

```
kubectl delete -f deployment.yaml
kubectl delete -f service.yaml
```